

KUBERNETES – TASK -02

- =====
1. **Create a Simple Pod Using YAML Task:** Write a YAML file to create a Pod named firstpod with an nginx container. Verify the Pod creation using `kubectl get pods` and check the logs of the container using `kubectl logs firstpod`.

vi firstpod.yaml

apiVersion: v1

kind: Pod

metadata:

name: firstpod

spec:

containers:

- name: nginx-container

image: nginx

ports:

- containerPort: 80

Deploy

- `kubectl apply -f firstpod.yaml`

Verify

- `kubectl get pods`

Check Logs

- `kubectl logs firstpod`

```
aws
Search [Alt+S]
IAM VPC EC2 S3 Route 53 Certificate Manager
ubuntu@k8s-master:~$ kubectl get nodes
NAME STATUS ROLES AGE VERSION
k8s-master Ready control-plane 24h v1.29.15
worker01 Ready <none> 24h v1.29.15
worker02 Ready <none> 23h v1.29.15
ubuntu@k8s-master:~$ kubectl get pods
NAME READY STATUS RESTARTS AGE
nginx 1/1 Running 1 (38m ago) 23h
ubuntu@k8s-master:~$ vi firstpod.yaml
ubuntu@k8s-master:~$ cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
  - name: nginx-container
    image: nginx
    ports:
    - containerPort: 80
ubuntu@k8s-master:~$ kubectl apply -f firstpod.yaml
error: error parsing firstpod.yaml: error converting YAML to JSON: yaml: line 7: did not find expected '-' indicator
ubuntu@k8s-master:~$ vi firstpod.yaml
ubuntu@k8s-master:~$ kubectl apply -f firstpod.yaml
pod/firstpod created
ubuntu@k8s-master:~$ kubectl get pods
NAME READY STATUS RESTARTS AGE
firstpod 1/1 Running 0 27s
nginx 1/1 Running 1 (57m ago) 24h
ubuntu@k8s-master:~$ kubectl logs firstpod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
```

i-004f01f684ce82ec3 (k8s-master)
PublicIPs: 3.227.18.139 PrivateIPs: 172.31.4.85

2. Set Environment Variables in a Pod Task: Modify the YAML file to include environment variables myname: sabair and City: Hyderabad. Deploy the Pod and use kubectl exec <pod_name> -- env to check if the environment variables are set properly

vi firstpod.yaml

apiVersion: v1

kind: Pod

metadata:

name: firstpod

spec:

containers:

- name: nginx-container

image: nginx

ports:

- containerPort: 80

env:

- name: myname

value: "sabair"

- name: City

value: "Hyderabad"

Deploy the Pod

If the Pod already exists, first delete it:

kubectl delete pod firstpod

Then create it again:

```
kubectl apply -f firstpod.yaml
```

Check:

```
kubectl get pods
```

Wait until it shows:

```
firstpod 1/1 Running
```

Check the environment variables

Run:

```
kubectl exec firstpod -- env
```

You'll see many environment variables. Look for:

```
myname=sabair
```

```
City=Hyderabad
```

Easier way — check only your variables

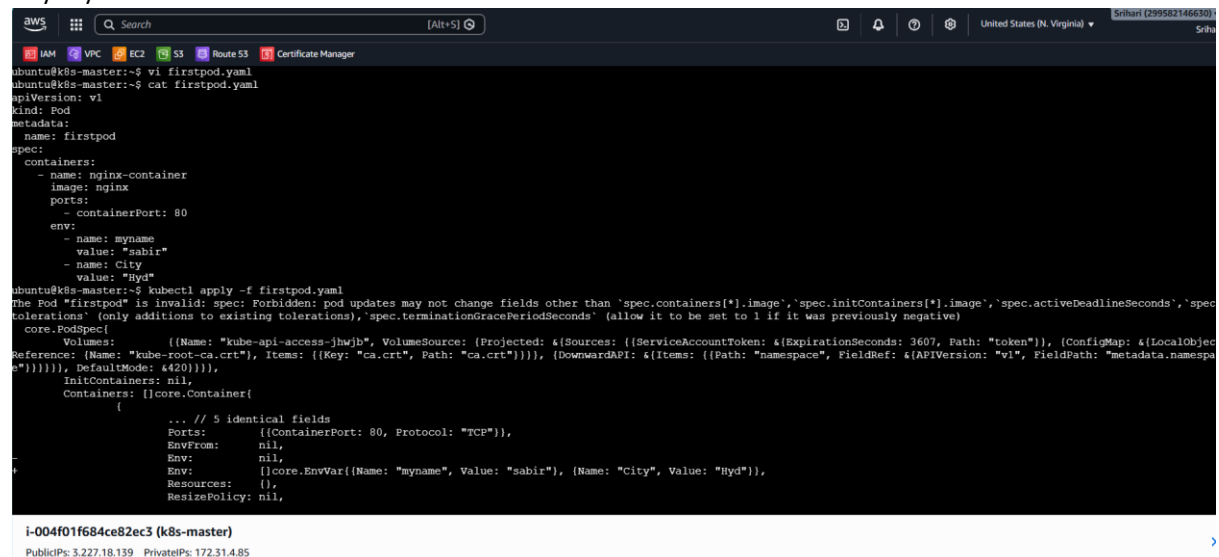
Instead of displaying everything:

```
kubectl exec firstpod -- env | grep -E 'myname|City'
```

Expected:

```
myname=sabair
```

```
City=Hyderabad
```



```
aws
Search [Alt+S]
IAM VPC EC2 S3 Route 53 Certificate Manager
ubuntu@k8s-master:~$ vi firstpod.yaml
ubuntu@k8s-master:~$ cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
  - name: nginx-container
    image: nginx
    ports:
    - containerPort: 80
    env:
    - name: myname
      value: "sabir"
    - name: City
      value: "Hyd"
ubuntu@k8s-master:~$ kubectl apply -f firstpod.yaml
The Pod "firstpod" is invalid: spec: Forbidden: pod updates may not change fields other than 'spec.containers[*].image', 'spec.initContainers[*].image', 'spec.activeDeadlineSeconds', 'spec.tolerations' (only additions to existing tolerations), 'spec.terminationGracePeriodSeconds' (allow it to be set to 1 if it was previously negative)
core.PodSpec
  Volumes:
    - (Name: "kube-api-access-jhwjb", VolumeSource: (Projected: &{Sources: &{ServiceAccountToken: &{ExpirationSeconds: 3607, Path: "token"}}, (ConfigMap: &{LocalObjectReference: (Name: "kube-root-ca.crt", Items: &{{Key: "ca.crt", Path: "ca.crt"}})}, (DownwardAPI: &{Items: &{{Path: "namespace", FieldRef: &{APIVersion: "v1", FieldPath: "metadata.namespace"}}}})}, DefaultMode: &{420}})),
    InitContainers: nil,
    Containers: &{core.Container{
      ... // 5 identical fields
      Ports: &{{ContainerPort: 80, Protocol: "TCP"}},
      EnvFrom: nil,
      Env: nil,
      EnvVar: &{core.EnvVar{Name: "myname", Value: "sabir"}, (Name: "City", Value: "Hyd")},
      Resources: {},
      ResizePolicy: nil,
```

i-004f01f684ce82ec3 (k8s-master)
PublicIPs: 3.227.18.139 PrivateIPs: 172.31.4.85

```
{
  ... // 5 identical fields
  Ports:      [(ContainerPort: 80, Protocol: "TCP")],
  EnvFrom:    nil,
  Env:        nil,
  Env:        [{}core.EnvVar{(Name: "myname", Value: "sabir"), (Name: "City", Value: "Hyd")}],
  Resources:  {},
  ResizePolicy: nil,
  ... // 14 identical fields
},
},
EphemeralContainers: nil,
RestartPolicy:      "Always",
... // 28 identical fields
}

ubuntu@k8s-master:~$ kubectl delete pod firstpod
pod "firstpod" deleted
ubuntu@k8s-master:~$ kubectl apply -f firstpod.yaml
pod/firstpod created
ubuntu@k8s-master:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
firstpod  1/1     Running   0           7s
nginx     1/1     Running   1 (75m ago)  24h
ubuntu@k8s-master:~$ kubectl exec firstpod -- env | grep -E 'myname|City'
myname=sabir
City=Hyd
ubuntu@k8s-master:~$
```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 3.227.18.139 PrivateIPs: 172.31.4.85

3. Deploy a Pod with Commands (Args) in YAML Task: Modify the YAML file to add args that instruct the container to sleep for 50 seconds. Deploy the Pod and use kubectl describe pod to verify the args are correctly passed to the container

Vi firstpod.yaml

apiVersion: v1

kind: Pod

metadata:

name: firstpod

spec:

containers:

- name: nginx-container

image: nginx

command: ["sleep"]

args: ["50"]

Delete the old Pod

Because your firstpod already exists:

kubectl delete pod firstpod

Then create the new one:

kubectl apply -f firstpod.yaml

Verify the Pod

Immediately run:

kubectl get pod firstpod

Verify the command and args

Run:

kubectl describe pod firstpod

```
ubuntu@k8s-master:~$ vi firstpod.yaml
ubuntu@k8s-master:~$ cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
  - name: nginx-container
    image: nginx
    command: ["sleep"]
    args: ["50"]
ubuntu@k8s-master:~$ kubectl delete pod firstpod
pod "firstpod" deleted
ubuntu@k8s-master:~$ kubectl apply -f firstpod.yaml
pod/firstpod created
ubuntu@k8s-master:~$ kubectl get pod firstpod
NAME          READY   STATUS    RESTARTS   AGE
firstpod      1/1     Running   0           14s
ubuntu@k8s-master:~$ kubectl describe pod firstpod
Name:          firstpod
Namespace:     default
Priority:       0
Service Account: default
Node:          worker02/172.31.4.12
Start Time:    Wed, 23 Sep 2026 09:15:11 +0000
Labels:        <none>
Annotations:   <none>
Status:        Running
IP:            10.244.2.5
IPs:
  IP: 10.244.2.5
Containers:
  nginx-container:
    Container ID:  containerd://0ae0fc6ed9214152966b11509e35c37b4cc11635c38b62287217422b3d8cd8a0
    Image:         nginx
    Image ID:      docker.io/library/nginx@sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f8cf2
    Port:         <none>
    Host Port:    <none>
    Command:
      sleep
    Args:
      50
    State:        Running
      Started:    Wed, 23 Sep 2026 09:15:11 +0000
    Ready:        True
    Restart Count: 0
```

4. **Create a Pod with Two Containers Task:** Create a YAML file to define a Pod with two nginx containers inside. Use kubectl exec to access both containers and verify that both containers can communicate through the same network (e.g., using telnet between them).

Step 1: Create the YAML

apiVersion: v1

kind: Pod

metadata:

name: twonginx

spec:

containers:

- name: nginx1

 - image: nginx

- name: nginx2

 - image: nginx

This creates:

Pod: twonginx

|

|— Container 1 → nginx1

|

|— Container 2 → nginx2

Step 2: Create the Pod

kubectl apply -f twocontainer.yaml

Then:

kubectl get pods

You should see:

twonginx 2/2 Running

2/2 means both containers are running.

Step 3: Enter container 1

kubectl exec -it twonginx -c nginx1 -- /bin/bash

Now you're inside nginx1.

Check its IP:

hostname -i

Then:

exit

Step 4: Enter container 2

kubectl exec -it twonginx -c nginx2 -- /bin/bash

Again:

hostname -i

You will see the **same IP address**.

That's the important part of this task.

Because **both containers are inside the same Pod**, so they share the same network.

If curl is available, you'll see Nginx HTML output.

```
ubuntu@k8s-master:~$ kubectl exec -it twonginx -c nginx1 -- /bin/bash
root@twonginx:/# hostname -i
10.244.2.7
root@twonginx:/# exit
exit
ubuntu@k8s-master:~$ kubectl exec -it twonginx -c nginx2 -- /bin/bash
root@twonginx:/# ls
bin boot dev docker-entrypoint.d docker-entrypoint.sh etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@twonginx:/# ll
bash: ll: command not found
root@twonginx:/# hostname -i
10.244.2.7
root@twonginx:/# kubectl exec twonginx -c nginx1 -- curl localhost:80
bash: kubectl: command not found
root@twonginx:/# exit
exit
command terminated with exit code 127
ubuntu@k8s-master:~$ kubectl exec twonginx -c nginx1 -- curl localhost:80
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total    Spent    Left     Speed

  0    0    0     0     0     0      0      0 --:--:-- --:--:-- --:--:--    0<!DOCTYPE html>

<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
```

PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85

5. **Set Up an Init Container in a Pod Task:** Modify the YAML to include an init container that sleeps for 30 seconds before the main containers start. Verify the init container's execution using `kubectl describe pod` and check the logs to confirm its completion.

What is an Init Container?

An **init container** runs before the main application containers.

In your case:

Pod starts



Init container



sleep 30 seconds



Init container completes



nginx1 + nginx2 start

Create initpod.yaml

Use this simple YAML:

```
apiVersion: v1
kind: Pod

metadata:
  name: initpod

spec:
  initContainers:
    - name: init-container
      image: busybox
      command: ["sh", "-c"]
      args:
        - echo "Init container started"; sleep 30; echo "Init container completed"

  containers:
    - name: nginx1
      image: nginx
      ports:
        - containerPort: 80

    - name: nginx2
      image: nginx
      command: ["/bin/sh", "-c"]
      args:
        - sed -i 's/listen      80;/listen      8080;/' /etc/nginx/conf.d/default.conf && nginx -g 'daemon off;'
      ports:
        - containerPort: 8080
```

Create the Pod

kubectl apply -f initpod.yaml

Immediately check:

kubectl get pod initpod

During the first 30 seconds, you may see:

NAME	READY	STATUS	RESTARTS	AGE
initpod	0/2	Init:0/1	0	5s

Init:0/1 means:

Init container has not completed yet.

After 30 seconds:

```
kubectl get pod initpod
```

You should see:

NAME	READY	STATUS	RESTARTS	AGE
initpod	2/2	Running	0	40s

Verify using describe

Run:

```
kubectl describe pod initpod
```

Look for:

Init Containers:

init-container:

State: Terminated

Reason: Completed

Exit Code: 0

Exit Code: 0 means the init container completed successfully.

Check Init Container logs

This is important for your task:

```
kubectl logs initpod -c init-container
```

You should see:

Init container started

Init container completed

IAM
VPC
EC2
S3
Route 53
Certificate Manager

```

apiVersion: v1
kind: Pod

metadata:
  name: initpod

spec:
  initContainers:
    - name: init-container
      image: busybox
      command: ["sh", "-c"]
      args:
        - echo "Init container started"; sleep 30; echo "Init container completed"

  containers:
    - name: nginx1
      image: nginx
      ports:
        - containerPort: 80

    - name: nginx2
      image: nginx
      command: ["/bin/sh", "-c"]
      args:
        - sed -i 's/listen      80;/listen      8080;/' /etc/nginx/conf.d/default.conf && nginx -g 'daemon off;'
      ports:
        - containerPort: 8080

```

```

ubuntu@k8s-master:~$ kubectl apply -f initpod.yaml
pod/initpod created
ubuntu@k8s-master:~$ kubectl get pod
NAME          READY   STATUS    RESTARTS   AGE
firstpod      0/1     CrashLoopBackOff   16 (4m32s ago)    75m
initpod       0/2     Init:0/1           0           14s
nginx         1/1     Running          2 (90m ago)       27h
twonginx      2/2     Running          0           22m
ubuntu@k8s-master:~$ kubectl get pod
NAME          READY   STATUS    RESTARTS   AGE
firstpod      1/1     Running   17 (5m49s ago)    76m
initpod       2/2     Running   0           91s
nginx         1/1     Running   2 (91m ago)       27h
twonginx      2/2     Running   0           23m
ubuntu@k8s-master:~$

```

i-004f01f684ce82ec3 (k8s-master)
PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85

```

Init Containers:
init-container:
  Container ID:   containerd://ee895b09d680a4e1f2c4f11d6075701c89768461c41627b85781b876fd196855
  Image:          busybox
  Image ID:       docker.io/library/busybox@sha256:fd7dc98638c8e305f4dc34e979f1c0fdfdcaeb0fbf8fcff77ae834b6da3d7e6e
  Port:           <none>
  Host Port:      <none>
  Command:
    sh
    -c
  Args:
    echo "Init container started"; sleep 30; echo "Init container completed"
  State:          Terminated
    Reason:        Completed
    Exit Code:     0

```

```

ubuntu@k8s-master:~$ kubectl logs initpod -c init-container
Init container started
Init container completed
ubuntu@k8s-master:~$

```

i-004f01f684ce82ec3 (k8s-master)
PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85

6. Run a Dry Run Command to Generate YAML Task: Use the `kubectl run nginx --image=nginx --dry-run=client -o yaml` command to generate a Pod YAML definition. Modify the generated YAML to suit specific requirements (e.g., labels or environment variables) and deploy it.

- `kubectl run nginx --image=nginx --dry-run=client -o yaml > nginx.yaml`
- `cat nginx.yaml`
- change the file to
- `vi nginx.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
    environment: dev
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
  env:
    - name: myname
      value: "sabair"
    - name: City
      value: "Hyderabad"
```

```
kubectl apply -f nginx.yaml
kubectl get pods
kubectl get pod nginx --show-labels
kubectl exec nginx -- env | grep -E 'myname|City'
```

```

ubuntu@k8s-master:~$ kubectl run nginx --image=nginx --dry-run=client -o yaml > nginx.yaml
ubuntu@k8s-master:~$ cat nginx.yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
    name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
ubuntu@k8s-master:~$ vi nginx.yaml
ubuntu@k8s-master:~$

```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85

```

apiVersion: v1
kind: Pod

metadata:
  name: nginx
  labels:
    app: nginx
    environment: dev

spec:
  containers:
  - name: nginx
    image: nginx
    env:
      - name: myname
        value: "sabair"
      - name: City
        value: "Hyderabad"

```

```

ubuntu@k8s-master:~$ kubectl apply -f nginx.yaml
pod/nginx created
ubuntu@k8s-master:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
firstpod  0/1     CrashLoopBackOff   23 (33s ago)    113m
initpod   2/2     Running    0           37m
nginx     1/1     Running    0           3m1s
twonginx  2/2     Running    0           59m
ubuntu@k8s-master:~$ kubectl get pod nginx --show-labels
NAME      READY   STATUS    RESTARTS   AGE   LABELS
nginx     1/1     Running    0           3m15s  app=nginx,environment=dev
ubuntu@k8s-master:~$ kubectl exec nginx -- env | grep -E 'myname|City'
myname=sabair
City=Hyderabad
ubuntu@k8s-master:~$

```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85

7. Use kubectl apply vs kubectl create Task: Create a YAML file to define a Pod. First, deploy it using kubectl create -f <file_name>.yml and then modify the YAML (e.g., change the image version). Use kubectl apply to redeploy and verify the difference between both commands.

Step 1 – Create YAML file

apiVersion: v1

kind: Pod

metadata:

name: nginx

spec:

containers:

- name: nginx

image: nginx:1.25

➤ `kubectl create -f hari.yaml`

If you run again → it fails (Already exists)

Step 2 – Modify YAML (Change Image)

Change:

➤ `image: nginx:1.27`

Now use:

➤ `kubectl apply -f firstpod.yaml`

apply updates existing resource
create only creates new resources

Verify:

➤ `kubectl describe pod nginx | grep image`

```
ubuntu@k8s-master:~$ vi hari.yaml
ubuntu@k8s-master:~$ kubectl create -f hari.yaml
pod/nginx created
ubuntu@k8s-master:~$ kubectl get pod nginx
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           13s
ubuntu@k8s-master:~$ kubectl describe pod nginx | grep Image
      Image:          nginx:1.25
      Image ID:       docker.io/library/nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.25
~
~
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.27
~
~
```

```
ubuntu@k8s-master:~$ vi hari.yaml
ubuntu@k8s-master:~$ kubectl create -f hari.yaml
Error from server (AlreadyExists): error when creating "hari.yaml": pods "nginx" already exists
Warning: resource pods/nginx is missing the kubect1.kubernetes.io/last-applied-configuration annotation which is required by kubectl apply. kubectl apply should only be used on resources created declaratively by either kubectl create --save-config or kubectl apply. The missing annotation will be patched automatically.
pod/nginx configured
ubuntu@k8s-master:~$ vi hari.yaml
ubuntu@k8s-master:~$ kubectl get pod nginx
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   1 (3m50s ago)  7m15s
ubuntu@k8s-master:~$ kubectl describe pod nginx | grep Image
      Image:          nginx:1.27
      Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
ubuntu@k8s-master:~$
```

kubectl create is used to create a resource, while kubectl apply is used to create or update a resource based on the YAML configuration.

It works only for image

Other wise it shows error...to solve that dlt and apply again

- 8. Edit an Existing Pod Configuration Task: Use kubectl edit pod <pod_name> to modify the running Pod's environment variables or image. After making the changes, verify if they took effect by checking the container logs or environment variables using kubectl exec.**

Edit Running Pod

➤ kubectl edit pod firstpod

Modify image or env variables.

⚠ Important:

Some fields (like image) are editable.

Some changes require Pod recreation.

Verify:

➤ kubectl describe pod firstpod

```
ubuntu@k8s-master:~$ kubectl edit pod nginx
pod/nginx edited
ubuntu@k8s-master:~$ kubectl describe pod hari
Error from server (NotFound): pods "hari" not found
ubuntu@k8s-master:~$ kubectl describe pod nginx
Name:          nginx
Namespace:     default
Priority:       0
Service Account: default
Node:          worker01/172.31.4.11
Start Time:    Wed, 23 Sep 2026 11:42:04 +0000
Labels:        <none>
Annotations:   <none>
Status:        Running
IP:            10.244.1.8
IPs:
  IP: 10.244.1.8
Containers:
  nginx:
    Container ID:   containerd://b7ec715facb2eb02d73b029fd0a9650c0d24964e132591daac92196c5b04f196
    Image:          nginx:1.25
    Image ID:       docker.io/library/nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e658666ce91f356ed7c72af059c
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Wed, 23 Sep 2026 12:14:40 +0000
    Last State:     Terminated
      Reason:       Completed
      Exit Code:    0
```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"nginx","namespace":"default"},"spec":{"containers":[{"image":"nginx:1.27","
creationTimestamp": "2026-09-23T11:42:04Z"}]}
  name: nginx
  namespace: default
  resourceVersion: "47601"
  uid: 5bcbadbb-b861-4d51-aa4f-daff0f4e0a09
spec:
  containers:
  - image: nginx:1.27
    imagePullPolicy: IfNotPresent
    name: nginx
    resources: {}
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: kube-api-access-9vqnv
      readOnly: true
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
  nodeName: worker01
  preemptionPolicy: PreemptLowerPriority
  priority: 0
  restartPolicy: Always
-- INSERT --
```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"nginx","namespace":"default"},"spec":{"containers":[{"image":"nginx
creationTimestamp": "2026-09-23T11:42:04Z"}]}
  name: nginx
  namespace: default
  resourceVersion: "47601"
  uid: 5bcbadbb-b861-4d51-aa4f-daff0f4e0a09
spec:
  containers:
  - image: nginx:1.25
    imagePullPolicy: IfNotPresent
    name: nginx
    resources: {}
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: kube-api-access-9vqnv
      readOnly: true
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
  nodeName: worker01
  preemptionPolicy: PreemptLowerPriority
  priority: 0
  restartPolicy: Always
-- INSERT --
```

9. Expose a Pod Using a Service Task: Create a YAML file to expose your firstpod using a Service (ClusterIP). Ensure that your service is exposing the Pod on port 80 and verify it using kubectl get svc.

Step 1: Modify hari.yaml

Change it to:

apiVersion: v1

kind: Pod

metadata:

name: nginx

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx:1.27

ports:

- containerPort: 80

Save:

Esc

:wq

Enter

Step 2: Recreate the Pod

Because labels are not something you should rely on changing directly on your existing Pod for this exercise:

```
kubectl delete pod nginx
```

```
kubectl apply -f hari.yaml
```

Check:

```
kubectl get pod nginx --show-labels
```

You should see:

app=nginx

Step 3: Create the Service YAML

```
vi nginx-service.yaml
```

Put:

apiVersion: v1

kind: Service

metadata:

name: nginx-service

spec:

type: ClusterIP

selector:

app: nginx

ports:

- port: 80

targetPort: 80

Save and apply:

kubectl apply -f nginx-service.yaml

Step 4: Verify

kubectl get svc

You should see:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
nginx-service	ClusterIP	10.96.x.x	<none>	80/TCP

Then check:

kubectl describe svc nginx-service

Look for:

Port: 80/TCP

TargetPort: 80/TCP

Endpoints: 10.244.x.x:80

what is the connection bw labels and service?

The connection is very simple:

Service uses labels to find the Pods to which it should send traffic.

Think of **labels as an address/tag** and the **Service as a traffic manager**.

```
IAM VPC EC2 S3 Route 53 Certificate Manager
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
~

ubuntu@k8s-master:~$ kubectl delete pod nginx
kubectl apply -f hari.yaml
pod "nginx" deleted
pod/nginx created
ubuntu@k8s-master:~$ kubectl get pod nginx --show-labels
NAME READY STATUS RESTARTS AGE LABELS
nginx 1/1 Running 0 9s app=nginx
ubuntu@k8s-master:~$ vi nginx-service.yaml
ubuntu@k8s-master:~$ kubectl apply -f nginx-service.yaml
service/nginx-service created
ubuntu@k8s-master:~$ kubectl get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 29h
nginx NodePort 10.96.199.90 <none> 80:30986/TCP 28h
nginx-service ClusterIP 10.111.70.94 <none> 80/TCP 6s
ubuntu@k8s-master:~$ kubectl describe svc nginx-service
Name: nginx-service
Namespace: default
Labels: <none>
Annotations: <none>
Selector: app=nginx
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.111.70.94
IPs: 10.111.70.94
Port: <unset> 80/TCP
TargetPort: 80/TCP
Endpoints: 10.244.2.8:80
Session Affinity: None
Events: <none>
ubuntu@k8s-master:~$ █

i-004f01f684ce82ec3 (k8s-master)
PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85
```

```
ubuntu@k8s-master:~$ kubectl run testpod --image=curlimages/curl -it --rm -- sh
If you don't see a command prompt, try pressing enter.
~ $
~ $ curl http://nginx-service
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

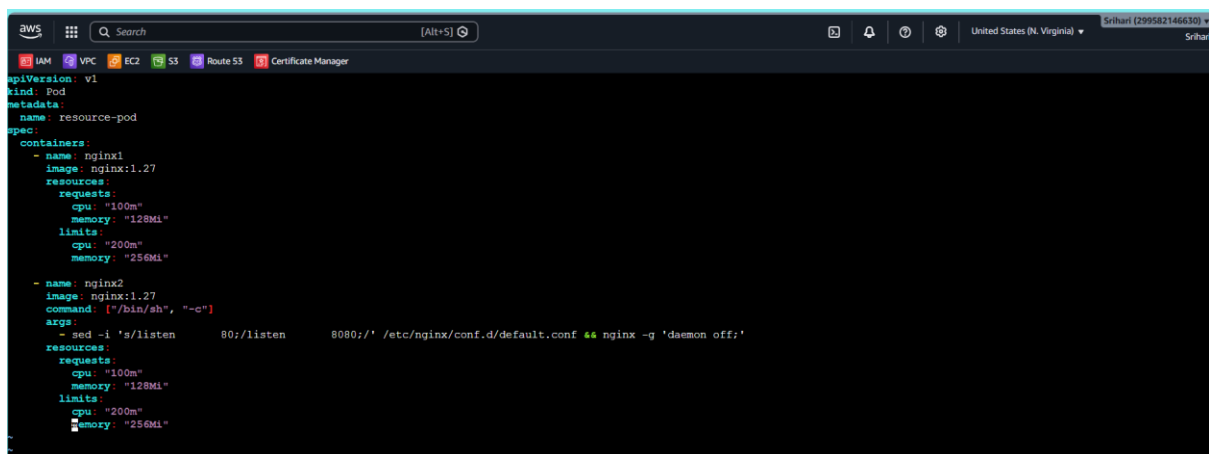
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
~ $ █

i-004f01f684ce82ec3 (k8s-master)
PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85
```

10. Pod with Resource Limits and Requests Task: Add resource requests and limits to the containers in your YAML file. Specify CPU and memory requests/limits for both containers and deploy the Pod. Use kubectl describe pod to verify if the resource configurations are correctly applied.

1. Create YAML

vi resource-pod.yaml



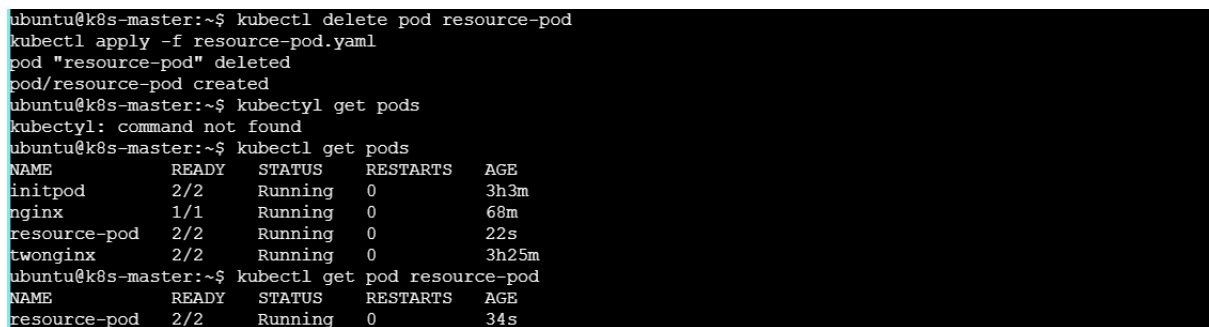
```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
  - name: nginx1
    image: nginx:1.27
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "200m"
        memory: "256Mi"
  - name: nginx2
    image: nginx:1.27
    command: ["/bin/sh", "-c"]
    args:
      - sed -i 's/listen      80;/listen      8080;/' /etc/nginx/conf.d/default.conf && nginx -g 'daemon off;'
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "200m"
        memory: "256Mi"
```

Deploy the Pod

kubectl apply -f resource-pod.yaml

Check:

kubectl get pod resource-pod



```
ubuntu@k8s-master:~$ kubectl delete pod resource-pod
kubectl apply -f resource-pod.yaml
pod "resource-pod" deleted
pod/resource-pod created
ubuntu@k8s-master:~$ kubectl get pods
kubectl: command not found
ubuntu@k8s-master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
initpod       2/2     Running   0           3h3m
nginx         1/1     Running   0           68m
resource-pod  2/2     Running   0           22s
twonginx      2/2     Running   0           3h25m
ubuntu@k8s-master:~$ kubectl get pod resource-pod
NAME          READY   STATUS    RESTARTS   AGE
resource-pod  2/2     Running   0           34s
```

Verify resource configuration

Run:

kubectl describe pod resource-pod

```
Containers:
  nginx1:
    Container ID:   containerd://28565db5a9186025249a6b771e04ec3af07a5Feb5c006d99122dc2fe4f3178ad
    Image:          nginx:1.27
    Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Wed, 23 Sep 2026 13:33:44 +0000
    Ready:         True
    Restart Count:  0
    Limits:
      cpu:         200m
      memory:      256Mi
    Requests:
      cpu:         100m
      memory:      128Mi
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-nzgmq (ro)
  nginx2:
    Container ID:   containerd://295b3dbc65d22c68f233a3163e59b4c2b61ebdbaff97c258a6053e1c6e079f7d
    Image:          nginx:1.27
    Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port:          <none>
    Host Port:     <none>
    Command:
      /bin/sh
      -c
    Args:
      sed -i 's/listen      80;/listen      8080;/' /etc/nginx/conf.d/default.conf && nginx -g 'daemon off;'
    State:         Running
      Started:     Wed, 23 Sep 2026 13:33:44 +0000
    Ready:         True
    Restart Count:  0
    Limits:
      cpu:         200m
      memory:      256Mi
    Requests:
      cpu:         100m
      memory:      128Mi
    Environment:   <none>
```

You can directly filter it:

kubectl describe pod resource-pod | grep -A6 -E "Limits:|Requests:"

```
ubuntu@k8s-master:~$ kubectl describe pod resource-pod | grep -A6 -E "Limits:|Requests:"
Limits:
  cpu:         200m
  memory:      256Mi
Requests:
  cpu:         100m
  memory:      128Mi
  Environment: <none>
  Mounts:
    /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-nzgmq (ro)
nginx2:
Limits:
  cpu:         200m
  memory:      256Mi
Requests:
  cpu:         100m
  memory:      128Mi
  Environment: <none>
  Mounts:
    /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-nzgmq (ro)
Conditions:
```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 3.218.145.134 PrivateIPs: 172.31.4.85

What are Requests and Limits?

Think of it like **booking a hotel room** 🏨.

Request = minimum resources needed

requests:

cpu: "100m"

memory: "128Mi"

Kubernetes says:

"I need at least 100m CPU and 128Mi memory to schedule this container."

The scheduler uses **requests** when deciding which worker node has enough resources.

Limit = maximum resources allowed

limits:

```
cpu: "200m"  
memory: "256Mi"
```

Means:

"This container can use up to 200m CPU and 256Mi memory."

CPU

100m = 0.1 CPU

200m = 0.2 CPU

So our Pod has:

Total requests:

CPU = 200m

Memory = 256Mi

Total limits:

CPU = 400m

Memory = 512Mi

Because there are two containers.

Easy interview answer

Requests specify the minimum CPU and memory required by a container and are used by the Kubernetes scheduler to place the Pod on a suitable node. Limits specify the maximum CPU and memory the container can use.